

Agentic Architect Challenge

Part 1: System Design & Critical Thinking

Prompt: "Design an agentic system that automates the processing of customer support emails. The system must:

1. Classify emails into multiple categories (e.g., Billing, Technical, Feedback).
2. Draft responses based on a provided internal knowledge base (PDFs/FAQs).
3. Route critical technical issues to human agents.
 - a. Route emails to a human agent if any of the following conditions are met: the email mentions data loss, service outage, or a security breach; the customer has contacted support more than 3 times in 7 days. Your system should flag these before drafting a response.
4. Ensure the system does not 'hallucinate' facts about our refund policy."

Part 2: Technical Implementation

Prompt: "This script is meant to scrape a website and summarize the content, but it fails on complex pages or long content. Identify the bottleneck, fix the scraping logic, and implement a guardrail to ensure the summary is concise."

Part 3: Practical Evaluation

Prompt: "Build a simple 'Agent' that can answer questions based on a single sample document (which you provide).

Requirements:

- Use an open-source framework (e.g., LangGraph, CrewAI, or raw OpenAI/Gemini API calls).
- Maintain 'conversation memory' (the agent must remember the user's name or previous context).
- Implement a 'tool' (e.g., a calculator or web search) that the agent must decide to call only when necessary."

Submission Guidelines

Please submit your work in the following format:

- A link to your GitHub repository containing all source code and documentation.
- A brief (1-page) PDF document explaining your system architecture, the trade-offs you made, and how you addressed potential failure points.
- Instructions for running your local environment (e.g., dependencies, API key requirements).
- You are required to present your system architecture during the interview.

Success Criteria

Your submission will be evaluated based on:

- System Reliability: Ability to handle errors gracefully and maintain performance under load.
- Code Quality: Adherence to best practices (e.g., modularity, documentation, and clean code principles).
- Agentic Logic: Correct use of planning/reasoning patterns rather than simple script execution.
- Operational Thinking: Consideration for logging, testing, and observability (AgentOps).